

Incorporating nested if statements into my compiler involves first the parser that identifies the grammar production rule $\text{IfStmt} \rightarrow \text{IF '(' Expr ')'} \text{ Stmt } [\text{ELSE Stmt}]$ at once, where the Stmt could be another IfStmt. This creation of a tree structure also places the inner ifs as children of the outer if-statement bodies. As a result, the parser creates an AST with IfNode structures that unit the condition expression, the statement of the then-branch, and the optional statement of the else-branch containing pointers each. The semantic analyzer then walks through this AST, evaluating the type of each condition expression and checking that it results in the boolean type, and then analyzing the statements of the then-branch and the else-branch recursively while keeping track of the symbol tables for local variable declarations if any in the branches. The compiler, during the code generation phase, makes use of a label-generation technique to assign distinct labels for the branches of each if statement (e.g., L1_then, L1_else, L1_end for the outer; L2_then, L2_else, L2_end for the nested if). The compiler generates code for each IfNode, which: (1) evaluates the condition result into a temporary register, (2) sends a conditional jump instruction (JZ or IF_FALSE) to the else-label if the condition is false, (3) calls recursively the generation of code for then-branch (which might include another complete if-statement along with its own labels), (4) generates an unconditional jump to the end-label, (5) marks the else-label, (6) continues with the recursive generation of code for the else-branch if exists, and (7) marks the end-label. Such a recursive approach, combined with unique label generation, guarantees that the nested if statements will be executed according to the programmed logic, given that the inner conditions will be fully processed first before the outer ones, and through the structured label hierarchy, every else is properly associated with its corresponding if.